第十三课预习报告

addaword.c

- 作用: 演示如何使用 `fprintf()`、`fscanf()` 和 `rewind()` 对文件进行追加写入和读取。
- 功能: 程序打开（或创建）名为 `wordy` 的文件，允许用户输入单词追加到文件中，直到输入以 `#` 开头的行结束，然后显示文件的全部内容。
- 演示效果:

```c
/* addaword.c -- uses fprintf(), fscanf(), and rewind() */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 41

int main(void)
{
    FILE *fp;
    char words[MAX];

    if ((fp = fopen("wordy", "a+")) == NULL)
    {
        fprintf(stdout,"Can't open \"wordy\" file.\n");
        exit(EXIT_FAILURE);
    }

    puts("Enter words to add to the file; press the #");
    puts("key at the beginning of a line to terminate.");
    while ((fscanf(stdin,"%40s", words) == 1)  && (words[0] != '#'))
        fprintf(fp, "%s\n", words);

    puts("File contents:");
    rewind(fp);           /* go back to beginning of file */
    while (fscanf(fp,"%s",words) == 1)
        puts(words);
    puts("Done!");
    if (fclose(fp) != 0)
        fprintf(stderr,"Error closing file\n");

```

```
    return 0;
}
```

```
gcc addaword.c -o addaword && echo "apple`nbanana`n#`n" | ./addaword
Enter words to add to the file; press the #
key at the beginning of a line to terminate.
File contents:
apple
banana
Done!
```

append.c

- 作用: 演示文件追加操作，将多个源文件内容追加到目标文件中，并显示追加后的结果。
- 功能: 程序允许用户输入目标文件名，然后逐个输入源文件名，将源文件内容复制追加到目标文件末尾。避免同文件自我追加并支持多次输入。
- 演示效果:

```c
/* append.c -- appends files to a file */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define BUFSIZE 4096
#define SLEN 81
void append(FILE *source, FILE *dest);
char * s_gets(char * st, int n);

int main(void)
{
    FILE *fa, *fs;  // fa for append file, fs for source file
    int files = 0;  // number of files appended
    char file_app[SLEN];  // name of append file
    char file_src[SLEN];  // name of source file
    int ch;

    puts("Enter name of destination file:");
    s_gets(file_app, SLEN);
    if ((fa = fopen(file_app, "a+")) == NULL)
    {
        fprintf(stderr, "Can't open %s\n", file_app);
        exit(EXIT_FAILURE);
```

```
    }
    if (setvbuf(fa, NULL, _IOFBF, BUFSIZE) != 0)
    {
        fputs("Can't create output buffer\n", stderr);
        exit(EXIT_FAILURE);
    }
    puts("Enter name of first source file (empty line to quit):");
    while (s_gets(file_src, SLEN) && file_src[0] != '\0')
    {
        if (strcmp(file_src, file_app) == 0)
            fputs("Can't append file to itself\n",stderr);
        else if ((fs = fopen(file_src, "r")) == NULL)
            fprintf(stderr, "Can't open %s\n", file_src);
        else
        {
            if (setvbuf(fs, NULL, _IOFBF, BUFSIZE) != 0)
            {
                fputs("Can't create input buffer\n",stderr);
                continue;
            }
            append(fs, fa);
            if (ferror(fs) != 0)
                fprintf(stderr,"Error in reading file %s.\n",
                        file_src);
            if (ferror(fa) != 0)
                fprintf(stderr,"Error in writing file %s.\n",
                        file_app);
            fclose(fs);
            files++;
            printf("File %s appended.\n", file_src);
            puts("Next file (empty line to quit):");
        }
    }
    printf("Done appending. %d files appended.\n", files);
    rewind(fa);
    printf("%s contents:\n", file_app);
    while ((ch = getc(fa)) != EOF)
        putchar(ch);
    puts("Done displaying.");
    fclose(fa);

    return 0;
}

```

```c
void append(FILE *source, FILE *dest)
{
    size_t bytes;
    static char temp[BUFSIZE]; // allocate once

    while ((bytes = fread(temp,sizeof(char),BUFSIZE,source)) > 0)
        fwrite(temp, sizeof (char), bytes, dest);
}

char * s_gets(char * st, int n)
{
    char * ret_val;
    char * find;

    ret_val = fgets(st, n, stdin);
    if (ret_val)
    {
        find = strchr(st, '\n');   // look for newline
        if (find)                  // if the address is not NULL,
            *find = '\0';          // place a null character there
        else
            while (getchar() != '\n')
                continue;
    }
    return ret_val;
}
```

```
gcc append.c -o append && echo "output.txt`ninput.txt`n`n" | ./append
Enter name of destination file:
Enter name of first source file (empty line to quit):
File input.txt appended.
Next file (empty line to quit):
Done appending. 1 files appended.
output.txt contents:
The
fabulous
programmer
enchanted
the
large
apple
banana
```

```
Done displaying.
```

count.c

- 作用: 演示如何使用标准 I/O 函数读取文件并统计字符数。
- 功能: 程序接受一个文件名作为命令行参数，逐字符读取并输出到标准输出，同时统计文件的总字符数，最后显示统计结果。
- 演示效果:

```c
/* count.c -- using standard I/O */
#include <stdio.h>
#include <stdlib.h> // exit() prototype

int main(int argc, char *argv[])
{
    int ch;         // place to store each character as read
    FILE *fp;       // "file pointer"
    unsigned long count = 0;
    if (argc != 2)
    {
        printf("Usage: %s filename\n", argv[0]);
        exit(EXIT_FAILURE);
    }
    if ((fp = fopen(argv[1], "r")) == NULL)
    {
        printf("Can't open %s\n", argv[1]);
        exit(EXIT_FAILURE);
    }
    while ((ch = getc(fp)) != EOF)
    {
        putc(ch,stdout);  // same as putchar(ch);
        count++;
    }
    fclose(fp);
    printf("File %s has %lu characters\n", argv[1], count);

    return 0;
}
```

```
gcc count.c -o count && ./count addaword.c
/* addaword.c -- uses fprintf(), fscanf(), and rewind() */
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 41

int main(void)
{
    FILE *fp;
    char words[MAX];

    if ((fp = fopen("wordy", "a+")) == NULL)
    {
        fprintf(stdout,"Can't open \"wordy\" file.\n");
        exit(EXIT_FAILURE);
    }

    puts("Enter words to add to the file; press the #");
    puts("key at the beginning of a line to terminate.");
    while ((fscanf(stdin,"%40s", words) == 1)  && (words[0] != '#'))
        fprintf(fp, "%s\n", words);

    puts("File contents:");
    rewind(fp);           /* go back to beginning of file */
    while (fscanf(fp,"%s",words) == 1)
        puts(words);
    puts("Done!");
    if (fclose(fp) != 0)
        fprintf(stderr,"Error closing file\n");

    return 0;
}
```

randbin.c

- 作用: 演示如何使用二进制文件进行随机访问读写操作。
- 功能: 程序将 1000 个 double 数写入二进制文件，然后根据用户输入的索引，随机访问文件读取对应的数值并输出。
- 演示效果:

```c
/* randbin.c -- random access, binary i/o */
#include <stdio.h>
#include <stdlib.h>
```

```c
#define ARSIZE 1000

int main()
{
    double numbers[ARSIZE];
    double value;
    const char * file = "numbers.dat";
    int i;
    long pos;
    FILE *iofile;

    // create a set of double values
    for(i = 0; i < ARSIZE; i++)
        numbers[i] = 100.0 * i + 1.0 / (i + 1);
    // attempt to open file
    if ((iofile = fopen(file, "wb")) == NULL)
    {
        fprintf(stderr, "Could not open %s for output.\n", file);
        exit(EXIT_FAILURE);
    }
    // write array in binary format to file
    fwrite(numbers, sizeof (double), ARSIZE, iofile);
    fclose(iofile);
    if ((iofile = fopen(file, "rb")) == NULL)
    {
        fprintf(stderr,
                "Could not open %s for random access.\n", file);
        exit(EXIT_FAILURE);
    }
    // read selected items from file
    printf("Enter an index in the range 0-%d.\n", ARSIZE - 1);
    while (scanf("%d", &i) == 1 && i >= 0 && i < ARSIZE)
    {
        pos = (long) i * sizeof(double); // calculate offset
        fseek(iofile, pos, SEEK_SET);    // go there
        fread(&value, sizeof (double), 1, iofile);
        printf("The value there is %f.\n", value);
        printf("Next index (out of range to quit):\n");
    }
    // finish up
    fclose(iofile);
    puts("Bye!");

    return 0;
```

```
}
```

```
gcc randbin.c -o randbin && echo "0`n10`n999`n1000`n" | ./randbin
Enter an index in the range 0-999.
The value there is 1.000000.
Next index (out of range to quit):
The value there is 1000.090909.
Next index (out of range to quit):
The value there is 99900.001001.
Next index (out of range to quit):
Bye!
```

reducto.c

- 作用: 演示如何通过读取文件并输出每第三个字符来压缩文件内容。
- 功能: 程序接受一个文件名作为命令行参数，生成一个新文件（原文件名加 `.red` 后缀），其中只保留原文件中每第三个字符，从而减少文件大小。
- 演示效果:

```c
// reducto.c -- reduces your files by two-thirds!
#include <stdio.h>
#include <stdlib.h>    // for exit()
#include <string.h>    // for strcpy(), strcat()
#define LEN 40

int main(int argc, char *argv[])
{
    FILE  *in, *out;   // declare two FILE pointers
    int ch;
    char name[LEN];    // storage for output filename
    int count = 0;

    // check for command-line arguments
    if (argc < 2)
    {
        fprintf(stderr, "Usage: %s filename\n", argv[0]);
        exit(EXIT_FAILURE);
    }
    // set up input
    if ((in = fopen(argv[1], "r")) == NULL)
    {
```

```
        fprintf(stderr, "I couldn't open the file \"%s\"\n",
                argv[1]);
        exit(EXIT_FAILURE);
    }
    // set up output
    strncpy(name,argv[1], LEN - 5); // copy filename
    name[LEN - 5] = '\0';
    strcat(name,".red");            // append .red
    if ((out = fopen(name, "w")) == NULL)
    {                       // open file for writing
        fprintf(stderr,"Can't create output file.\n");
        exit(3);
    }
    // copy data
    while ((ch = getc(in)) != EOF)
        if (count++ % 3 == 0)
            putc(ch, out);  // print every 3rd char
    // clean up
    if (fclose(in) != 0 || fclose(out) != 0)
        fprintf(stderr,"Error in closing files\n");

    return 0;
}
```

```
gcc reducto.c -o reducto && ./reducto addaword.c
# 程序运行后会生成 addaword.red 文件，其中只包含 addaword.c 中每第三个字符
```

reverse.c

- 作用: 演示如何将文件内容按字符逆序输出。
- 功能: 程序读取指定文件，从文件末尾开始逐字符向前读取并输出，忽略 DOS 文本文件中的 `CTRL+Z` 和回车符。
- 演示效果:

```
/* reverse.c -- displays a file in reverse order */
#include <stdio.h>
#include <stdlib.h>
#define CNTL_Z '\032'   /* eof marker in DOS text files */
#define SLEN 81
int main(void)
{
```

```c
    char file[SLEN];
    char ch;
    FILE *fp;
    long count, last;

    puts("Enter the name of the file to be processed:");
    scanf("%80s", file);
    if ((fp = fopen(file,"rb")) == NULL)
    {                               /* read-only mode   */
        printf("reverse can't open %s\n", file);
        exit(EXIT_FAILURE);
    }

    fseek(fp, 0L, SEEK_END);        /* go to end of file */
    last = ftell(fp);
    for (count = 1L; count <= last; count++)
    {
        fseek(fp, -count, SEEK_END); /* go backward      */
        ch = getc(fp);
        if (ch != CNTL_Z && ch != '\r')  /* MS-DOS files */
            putchar(ch);
    }
    putchar('\n');
    fclose(fp);

    return 0;
}
```

```
gcc reverse.c -o reverse && echo "wordy" | ./reverse
Enter the name of the file to be processed:
banana
apple
large
the
enchanted
programmer
fabulous
The
```